

Fabric data agent Workshop

Prerequisites

- A paid F2 or higher Fabric capacity, or a Power BI Premium per capacity (P1 or higher) capacity with Microsoft Fabric enabled.
- Fabric tenant settings:
 - Enable cross-geo processing and cross-geo storing for AI based on requirements explained in Fabric data agent tenant settings. ([Data agent tenant settings](#))
 - Enable “Users can use Copilot and other features powered by Azure OpenAI” setting
- Power BI Pro license
- At least a Contributor role in a workspace
- Familiarity with Power BI, DAX, Python, SQL

Optional:

- [Power BI Desktop](#)
- PDF Viewer

Lab 1: Creating and optimizing data agents

Lab goal: This lab focuses on creating data agents in Fabric using a semantic model as a data source. Participants will learn how to create a data agent, how a data agent works, and how to configure and optimize it using best practices.

Resources required:

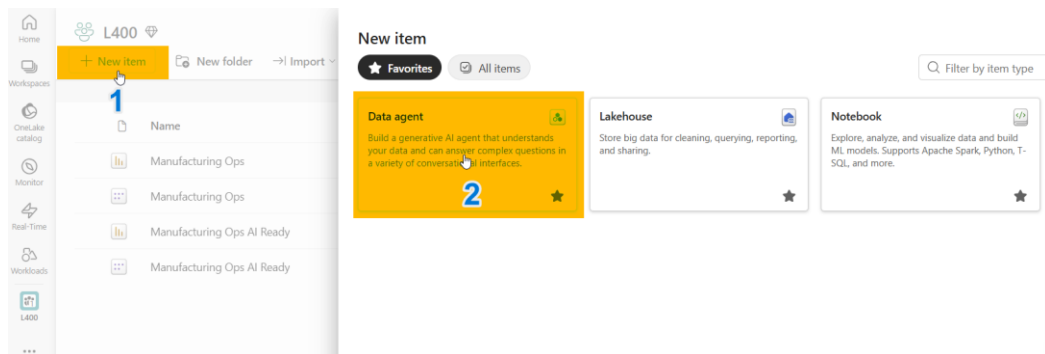
- A Fabric paid capacity with data agent tenant settings enabled
- A Power BI Pro license
- A workspace with at least workspace Contributor role

Step 1: Creating your first data agent

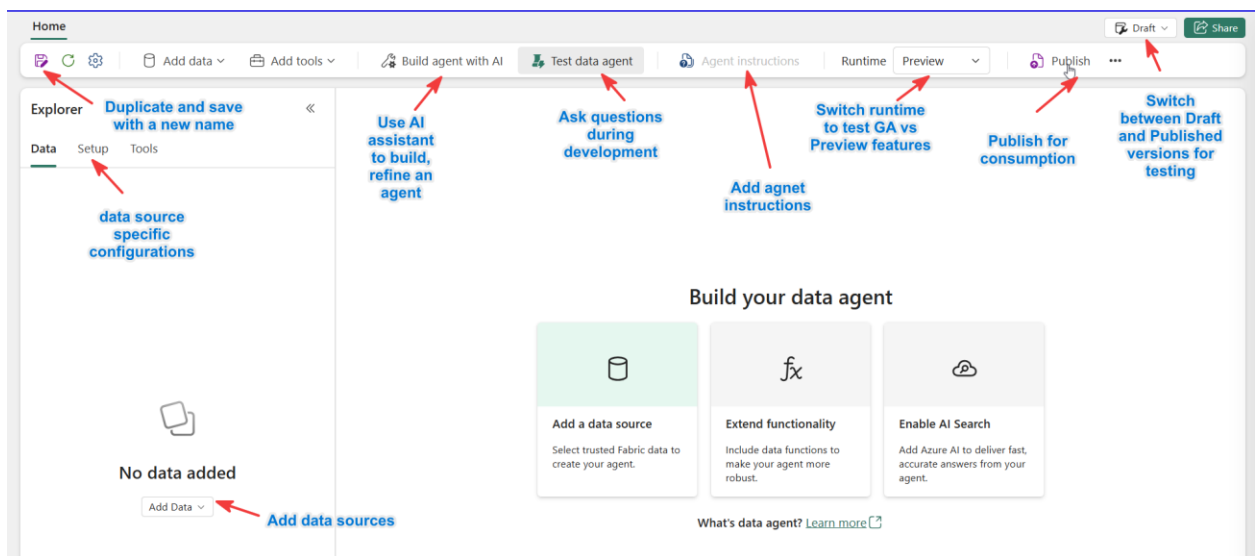
- In your lab workspace, confirm you have the following items: two semantic models, two Power BI reports and six notebooks in `getting-started-data-agents` folder

	Name
	BuildOpsRefData
	CreateMultiSourceDataAgent
	DataAgentGettingStarted
	EvaluateDataAgent
	InstallWorkshopAssets
	JudgeCalibration
	ManufacturingOps
	ManufacturingOps
	ManufacturingOpsAIReady
	ManufacturingOpsAIReady
	RefreshSemanticModel

- In the `getting-started-data-agents` folder, click `New item > Data agent` to create a data agent. You can also search for “data agent” in the search box.



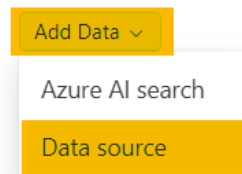
- Name the data agent MfgOps_DA_<YourInitials>
- Familiarize yourself with the user interface



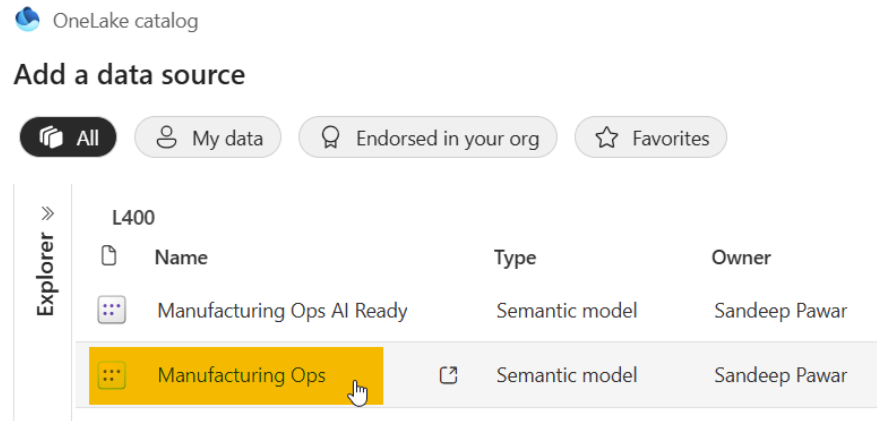
- In the Explorer pane, select Add Data > Data source



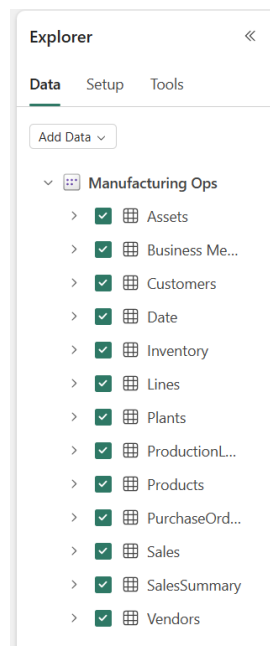
No data added



- In the OneLake catalog, select the **ManufacturingOps** semantic model from your workspace, then click Add to complete the selection.



- Select all the tables. The data agent will reason over these tables and objects to answer your questions.



- Ask your first question:

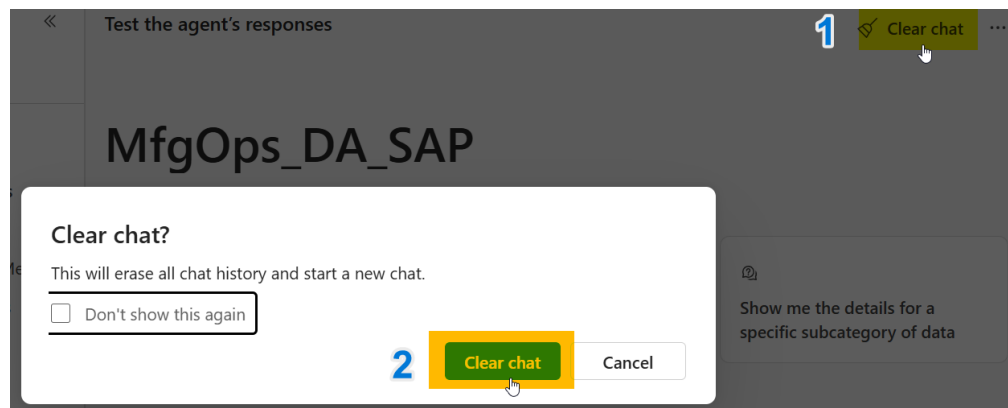
I am new this agent and the data. Tell me more about it and how to use it?

You will receive a response with a brief explanation and suggestions with sample questions you can ask. Test the agent by asking some of the example questions returned.

You can also ask the following questions:

- *Show me the production quantity for the last month*
- *List the top 5 products by total sales*
- *Give me a pivot table with product, total sales, quantity, and inventory*

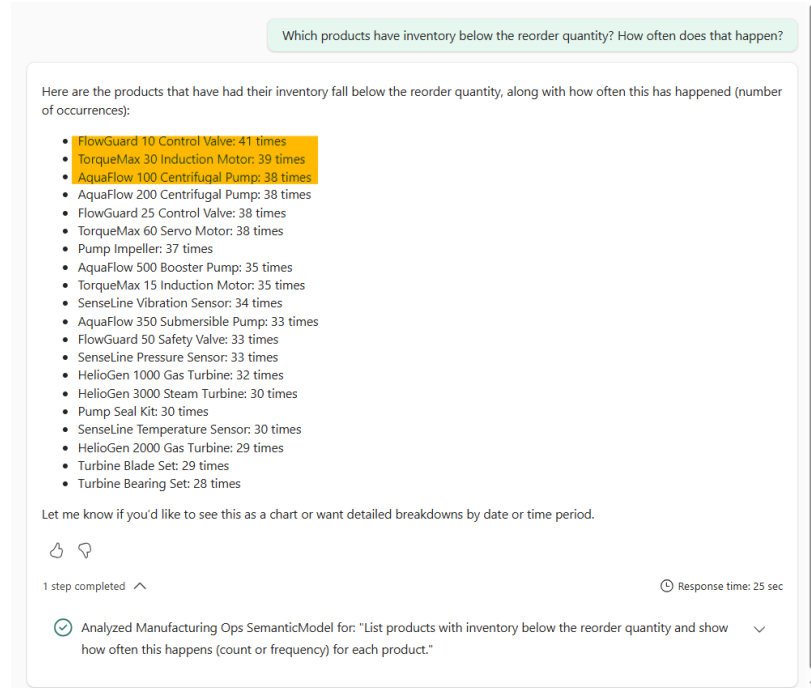
- Click **Clear chat** (top right corner) to clear the conversation history before proceeding. It is recommended to clear the chat between tests during development, so you can evaluate each configuration change in isolation.



Now let's ask a few more questions to learn about data agent capabilities.

Note that your responses may differ from the screenshots shown in this lab.

- Ask the following questions in sequence and observe the responses:
 - **Multi-part questions:**
Which products have inventory below the reorder quantity? How often does that happen?
The data agent may break down a question into multiple sub-questions or generate one single query to generate the answer. It may generate a visual answer as well.



○ **Query breakdown and context carry-over:**

For the top three products by frequency, show me the monthly trend as a bar chart

This question specifically asked about the top three products, which the data agent identified from the previous response. For those three products, the data agent generated a bar chart. Notice that the data agent executed three separate queries and combined the results to produce the final visual.

★**Tip:** The data agent keeps conversation context by default: phrases like “the top three” or “those products” refer back to your previous question. Use *Clear chat* whenever you want to test a question in complete isolation.

For the top three products by frequency, show me the monthly trend as a bar chart

Here is a bar chart showing the monthly trend of inventory falling below the reorder quantity for the top three products (FlowGuard 10 Control Valve, TorqueMax 30 Induction Motor, and AquaFlow 100 Centrifugal Pump). Each bar represents how many times the product's inventory was below its reorder level in that month.

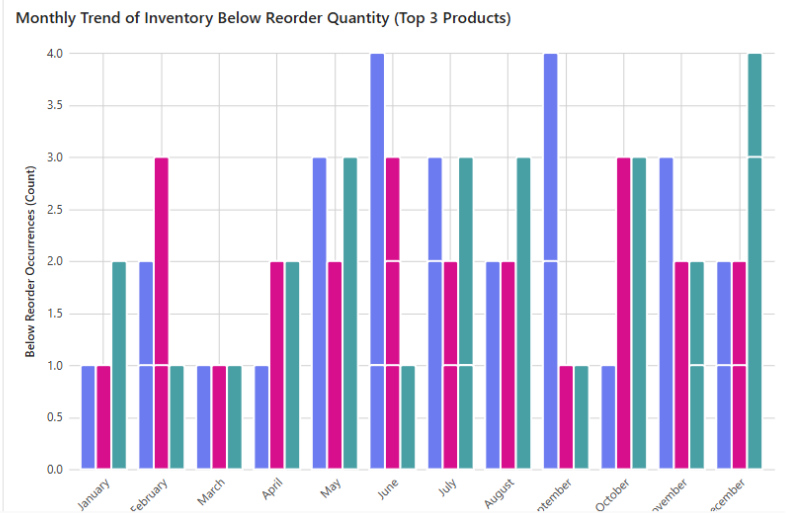
You'll be able to see which months experienced more inventory issues for each product. If you'd like the raw data or need the chart broken down differently (by year, for example), just let me know!



3 steps completed ✓

⌚ Response time: 37 sec

File(s) [report_specs_4.json](#) were generated as a result of the query.



- ✓ Analyzed Manufacturing Ops SemanticModel for: "Show the monthly occurrences where inventory for FlowGuard 10 Control Valve was below the reorder quantity." ✓
- ✓ Analyzed Manufacturing Ops SemanticModel for: "Show the monthly occurrences where inventory for TorqueMax 30 Induction Motor was below the reorder quantity." ✓
- ✓ Analyzed Manufacturing Ops SemanticModel for: "Show the monthly occurrences where inventory for AquaFlow 100 Centrifugal Pump was below the reorder quantity." ✓

○ **Domain-specific KPIs:**

For the bottom 20% of products by revenue, do we run into inventory issues? Why?

Click "1 step completed" to see how the data agent arrived at the answer.

1 step completed

Response time: 56 sec

Analyzed Manufacturing Ops SemanticModel for: "List the bottom 20% products by revenue and indicate whether they have occurrences of inventory falling below reorder quantity. Also, provide insights or reasons why these inventory issues may be happening for these low-revenue products."

Inspect the DAX query the data agent generated. Notice that, without any additional instructions, the data agent interpreted the vague “inventory issues” question using the schema and context of the data source, reusing existing inventory-related measures and creating new ones to answer the question and generate the response.

```

"Inventory Risk SKU Count",
[Inventory Risk SKU Count],
"Inventory Shortfall Qty",
[Inventory Below Reorder Qty],
"Has Inventory Below Reorder",
VAR _Risk = [Inventory Risk SKU Count]
RETURN
IF(_Risk > 0, "Yes", "No"),
"Avg Inventory Qty",
AVERAGE('Inventory'[Qty]),
"Avg Reorder Level",
AVERAGE('Inventory'[Reorder]),
"Avg InTransit Qty",
AVERAGE('Inventory'[InTransit]),
"Likely Inventory Issue Reason",
VAR _Risk2 = [Inventory Risk SKU Count]
VAR _AvgInv = AVERAGE('Inventory'[Qty])

```

Step 2: Improving and optimizing data agent responses

As we saw in the previous step, the data agent was able to generate natural language responses grounded in the semantic model data. To improve those responses further, we need to understand how the data agent works, how to spot common issues, and how to fix them, increasing accuracy and trust in the results. For each question below, ask it, then review these three outputs the data agent returns:

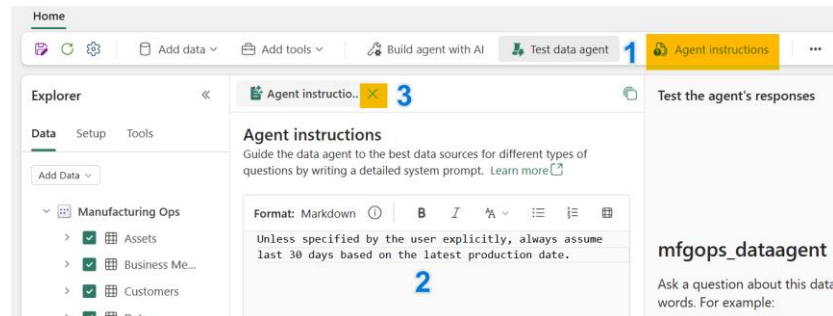
- Paraphrased question
- DAX query
- Final response

Clear the chat history before proceeding. In the following section, ask the specified question, expand the run step and observe the response.

Q1: What is our scrap %?

Observation: Inspect the DAX by expanding the run step. The data agent uses the existing [Scrap Rate %] measure but has to guess the time window, defaulting to either the latest date or the entire time period. The answer isn't wrong, but the assumed period isn't

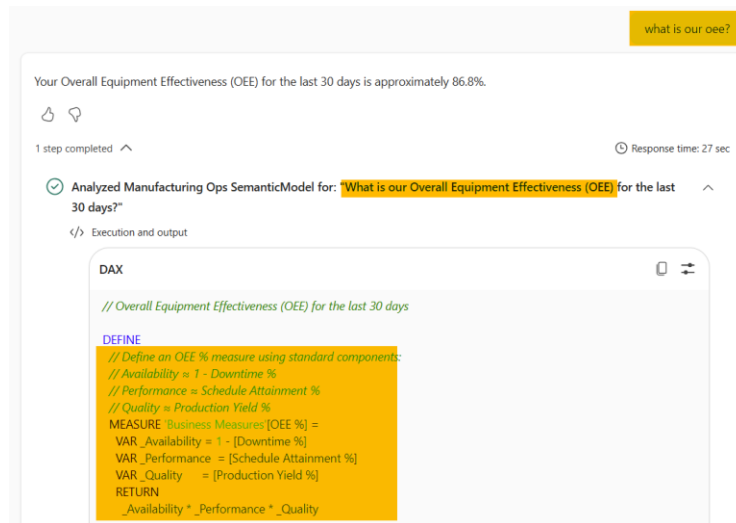
guaranteed to match what the user meant. In the next lab, we will remove this ambiguity by instructing the data agent to use a default time window, for example, the last 30 days. Agent instructions can be used to steer the agent behavior and responses.



Q2: What is our OEE?

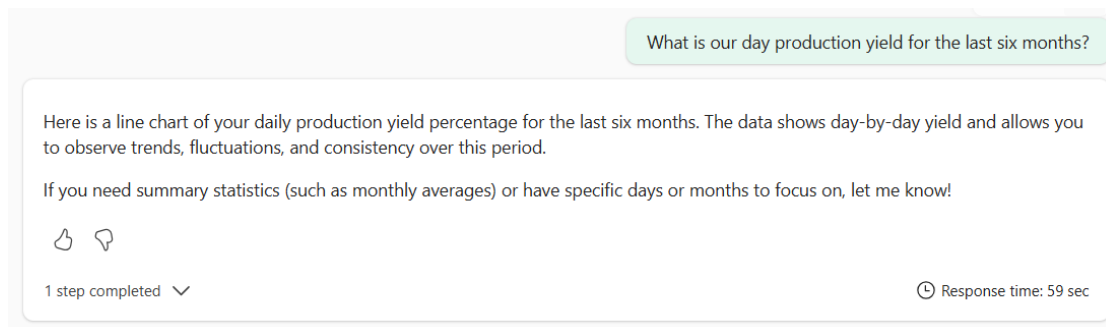
Observation: OEE isn't defined anywhere in the model or the agent configuration however it is a manufacturing industry term. Even so, the data agent infers from the schema context that OEE means Overall Equipment Effectiveness and returns the KPI by creating an ad hoc measure. Because this measure isn't defined in the model, the data agent may calculate it differently the next time the same question is asked, leading to inconsistent answers over time. This is why it's best practice to define governed measures with business-friendly names directly in the semantic model, so the data agent always uses the same definition. We'll configure this in the next lab.

★Tip: Rule of thumb: any KPI users will ask about by name (OEE, scrap rate, yield) should be a governed measure in the semantic model, not something the data agent has to infer on the fly.



Q3: What is our day production yield for the last six months?

Observation: Inspect the DAX. The data agent should ideally use the [prd_yld_day] measure, but results may vary by user: some may see the data agent correctly use [prd_yld_day], while others may see it instead calculate day production yield using the [Production Yield %] measure, applied per day over the last six months.



This semantic model also has a poorly named measure, [prd_yld_day], with a code comment noting that it calculates day production yield by excluding the night shift; in other words, yield for day shifts only (Day Production Yield). This is the measure the data agent should have used.

```
1 prd_yld_day = // Day Production yield, excluding Night shift.
2 DIVIDE(
3     SUMX(FILTER(ProductionLog, ProductionLog[Shift] <> "Night"), ProductionLog[Good]),
4     SUMX(FILTER(ProductionLog, ProductionLog[Shift] <> "Night"), ProductionLog[Qty])
5 )
```

★**Tip:** The data agent relies on the schema, schema descriptions, Prep data for AI configuration, and the agent instructions to answer questions about the semantic model; it does not read the DAX code or DAX comments. We can be fixed by adding an instruction such as “For day production yield, use the [prd_yld_day] measure” to Prep data for AI. We'll configure this in the next lab.

Q4: What were the TP sales last week?

Observation: This semantic model has no column, column value, or *measure that defines what “tp” means*. At this company, “TP” (or “tp”) is an acronym for Turbine-Pump sales, calculated by a cryptically named measure, [sls_amt_x]. This is a common pattern in many organizations, where a KPI or value has aliases or acronyms an agent has no way of knowing.

What were the tp sales for last week?

No sales data was found for the product 'tp' for last week. This may mean there were no sales recorded in that period or the product might not exist in the sales database. If you'd like, you can check across a different timeframe or for another product.

1 step completed 

 Response time: 62 sec

 Analyzed Manufacturing Ops SemanticModel for: "Show the total sales for the product 'tp' for last week." 

As mentioned earlier, the data agent uses semantic model context to answer questions. We can add a description to the [sls_amt_x] measure so the data agent can use it.

Home table

Business Measures

Description

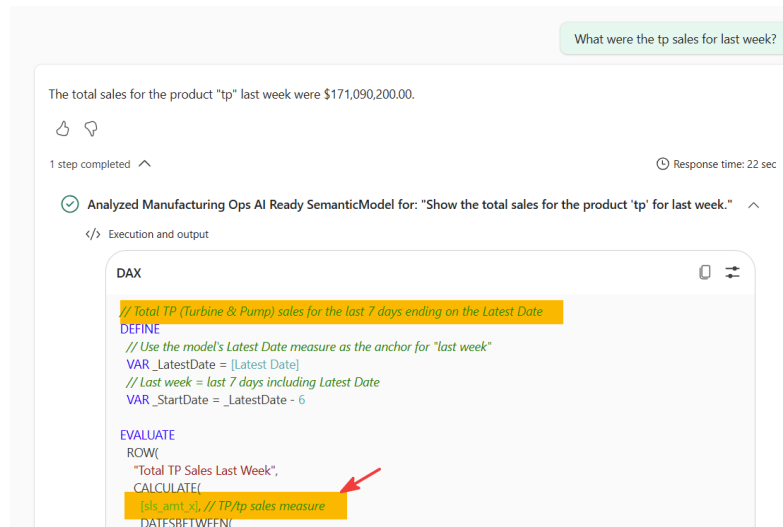
Sales amount for Turbines and Pumps only, also referred as TP or tp sales. If users asks about TP or tp sales, use this measure.

Business Measures

Ambiguous Names Demo

sls_amt_x

After adding the description, the data agent correctly maps “tp” to the measure and returns the right answer. We will test it in the next lab.



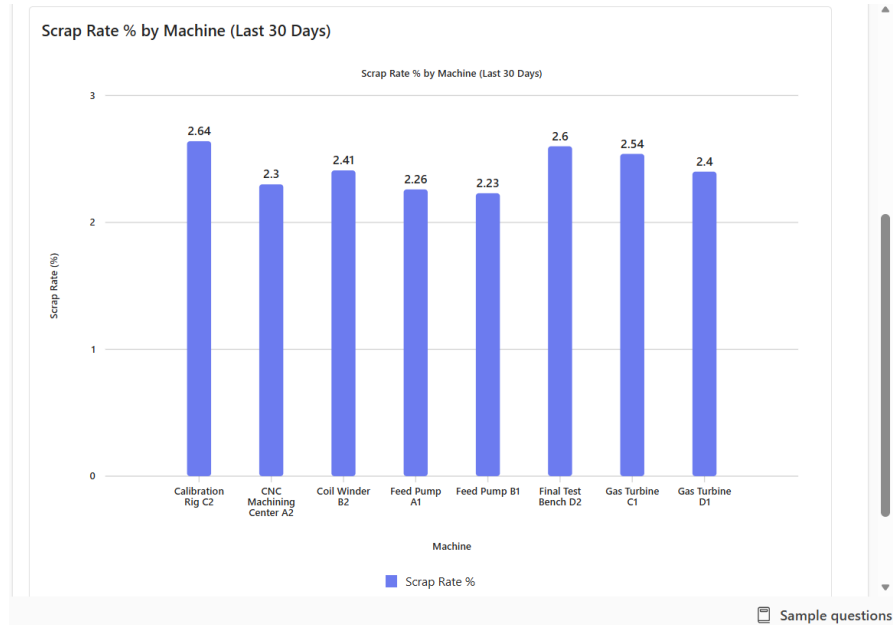
It is recommended to name tables, columns, and measures with business-friendly names, and to add descriptions based on domain and business context.

★Tip: Business-friendly names and descriptions are the primary context the data agent uses to resolve acronyms, internal codes, and ambiguous terms like “tp” or “oe.” Investing time here pays off across every question users ask, not just the one you’re testing. You can also instruct the agent ask for clarifying question if a term or acronym is not defined anywhere.

Q5: Show me the distribution of scrap rate by machine

Observation: The data agent assumes Machine maps to the Lines[asset] column and produces the visual.

```
SUMMARIZECOLUMNS(
  // Machine / asset name from the Lines table
  'Lines'[asset],
  // Apply last 30 days filter via the Date table
  _Last30Days,
  // Scrap rate percentage measure (scrap units ÷ production qty)
  "Scrap Rate %",
  [Scrap Rate %]
)
```



This looks correct, but at this company, any Machine-related question should use the `Lines[Manufacturing]` column instead. This is a company-specific context the agent will not know. This is a common scenario where a column name means one thing in the schema but something else to the business - a "Customer" column might really be a "user," or a "Product ID" a "SKU."

In this lab we tested how an agent can make assumptions about the user intent, data, context, industry, acronyms etc., if not configured correctly.

To get reliable results from any AI agent, you have to surface the right business context in the right configuration. In the next lab, we'll review these configuration options in detail and optimize the agent.

Step 3: Configuring and optimizing semantic model and data agent

In the previous steps, we saw how creating a trusted data agent requires analyzing the responses and configuring them correctly. In this step, we will look at all of these optimization levers and create a robust data agent based on defined scope and context.

In a data agent, you can add multiple data sources. However, it is always recommended to first identify and define the scope of the data agent. The scope could be related to a specific topic (for example, inventory management), a domain (for example, manufacturing operations), or an area (for example, a single plant or product line). You should define, configure and optimize based on that scope.

Scope

In our previous example, we selected all the tables and measures in the semantic model. However, in this exercise, we will limit the scope to only the questions related to Manufacturing Operations. Any tables, columns, measures and objects that are not required should be left out of the scope of this data agent.

Preparing the Semantic Model for AI

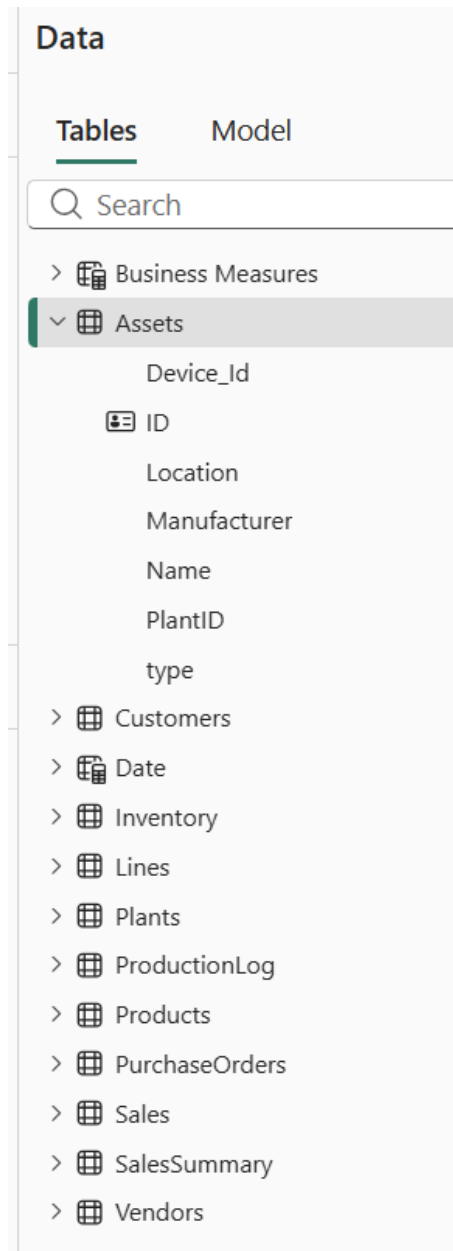
How a human consumes and uses the data is fundamentally different from how AI retrieves and uses the data. AI needs context from the human to use and retrieve the data correctly under the right context and guardrails. There are four primary levers:

- Semantic model foundation:
 - o Star schema
 - o RLS/CLS
 - o Schema names
- Business Logic:
 - o Measures
 - o Columns
- Semantic Metadata:
 - o Descriptions
 - o Synonyms
 - o Hierarchies
- AI Readiness and context:
 - o Prepare data for AI configurations
 - o Data agent configuration

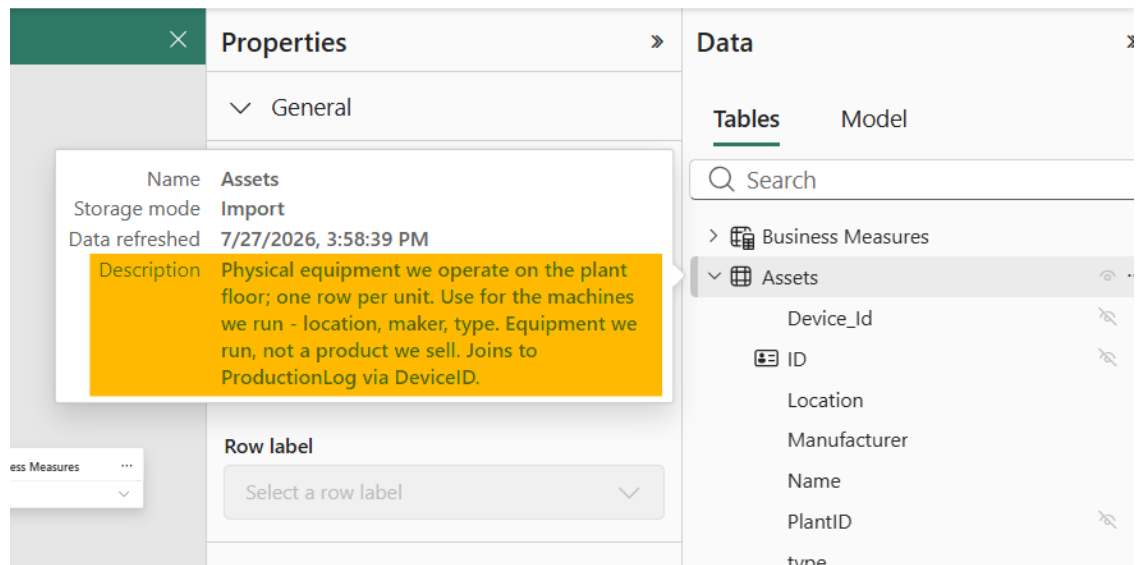
Semantic Model Configuration

Open the ManufacturingOpsAIRReady semantic model in your workspace. This semantic model has the following changes based on the tests we just performed:

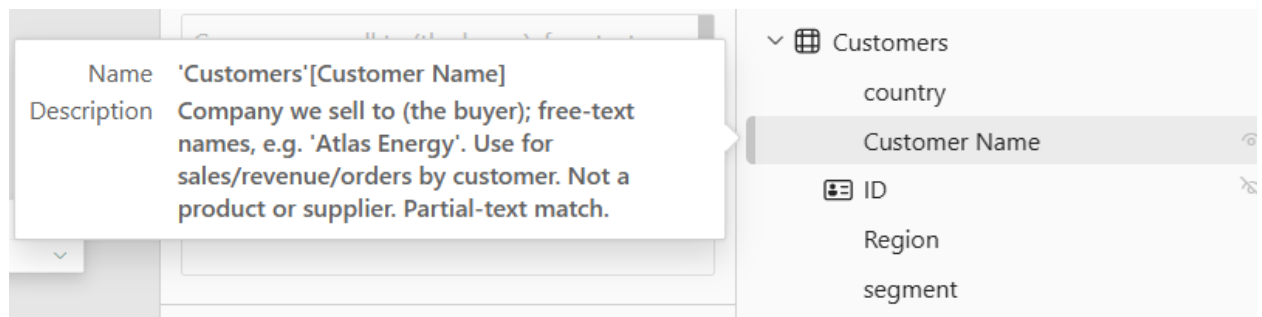
- All tables, column and measures have business friendly names

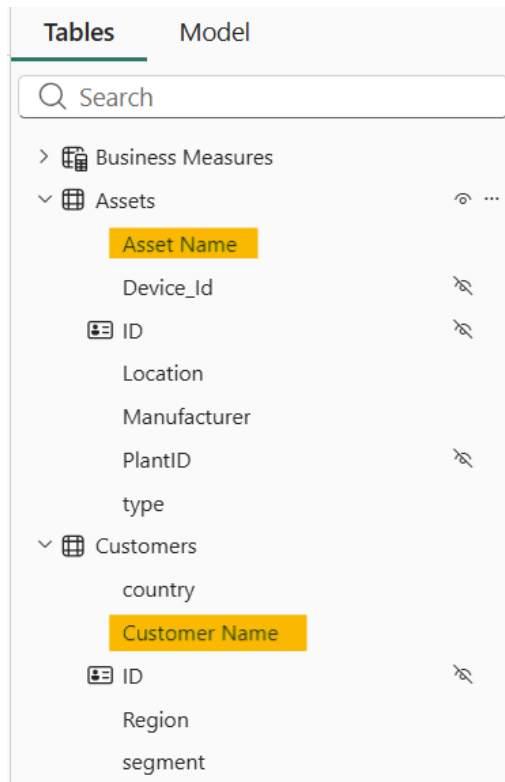


- Select any table, column or measure in the Data explorer pane. All objects have clear and concise descriptions that define the business context, when to use them. These descriptions are prioritized for AI retrieval rather than human information.

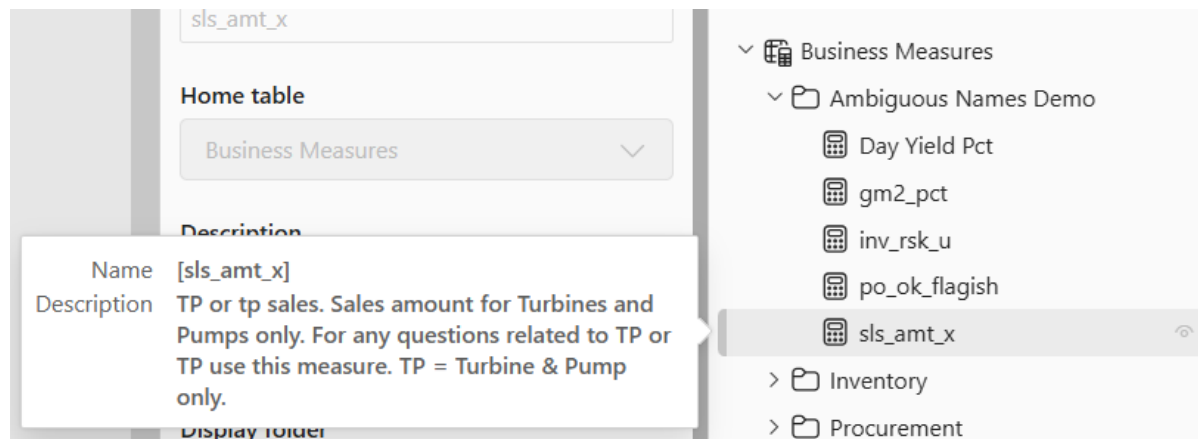


- Select the Customer[Customer Name] column. Instead of naming the column just [Name], it is more descriptive – [Customer Name]. This helps AI retrieve the objects consistently and quickly. Also notice the description includes an example value and when this column should be used. Descriptive names also help disambiguate between different entities such as Asset Name vs Customer Name





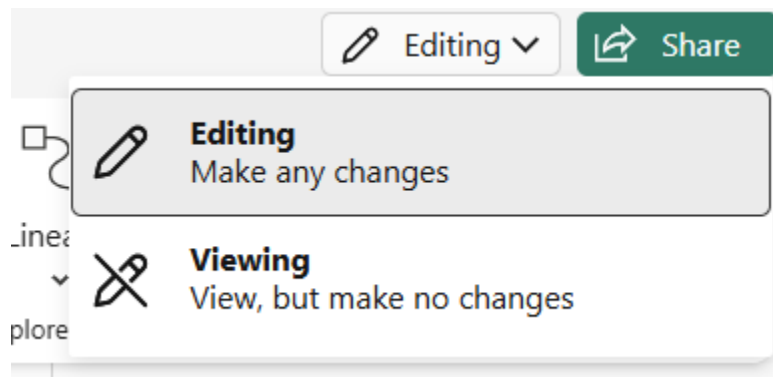
- Select [sls_amt_x] measure. Ideally, this measure should be appropriately renamed. However, often due to downstream dependencies, the objects cannot be renamed. In such cases, adding description can also help.



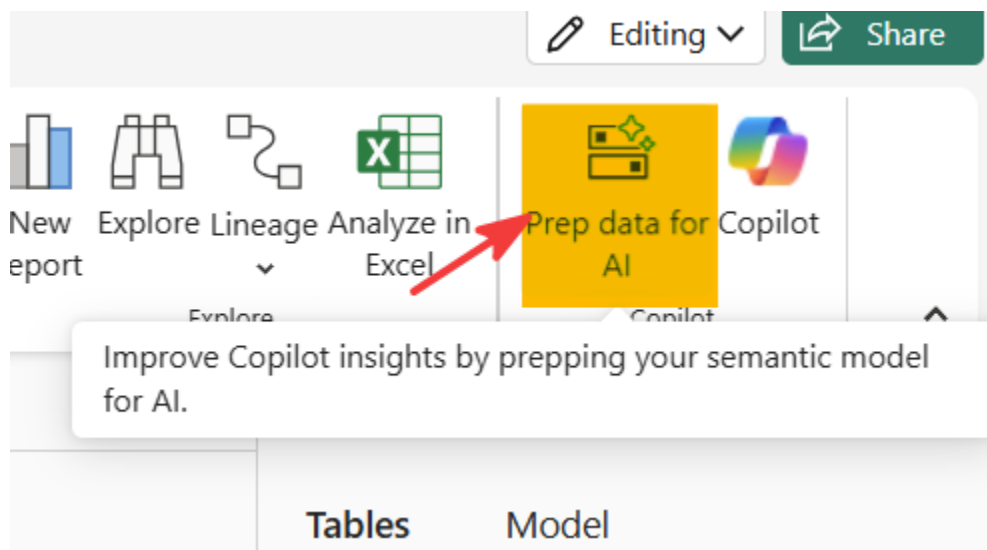
Prep data for AI

Prep data for AI features help optimize your semantic model for Copilot, data agent for improving accuracy, context, and relevance in AI-driven insights.

Step 1: First switch to Editing mode in the model view



Step 2: Select Prep data for AI button



Step 3: AI data schema: As mentioned earlier, for our data agent, we want to limit the scope to manufacturing operations related topics. An AI data schema lets semantic model authors define a focused subset of the model's schema for Copilot and data agent to prioritize when it generates responses.

Prep data for AI

Get started

| **Simplify the data schema**

Verified answers

Add AI instructions

Settings

Simplify the data schema (preview)

Improve response accuracy by deselecting fields Copilot doesn't schema saved with your semantic model. [Learn more](#)

🔍 Filter by keyword

- ▼ ☒  Semantic model
 - > ☒  Business Measures
 - > ☒  Assets
 - > ☐  Customers
 - > ☒  Date
 - > ☒  Inventory
 - > ☒  Lines
 - > ☒  Plants
 - > ☒  ProductionLog
 - > ☒  Products
 - > ☐  PurchaseOrders
 - > ☐  Sales
 - > ☐  SalesSummary
 - > ☐  Vendors

Step 4: Verified answers: Verified answers are human-approved, visual responses with predefined trigger phrases. Each verified answer includes one or more trigger phrases, a visual, and optional associated filters. In this case, for our question related to machines in the previous step, we define several prompts and select Manufacturer column. This lets the data agent know that for machine related questions, Manufacturer column should be used. Verified answers help improve accuracy, consistency and reduce latency.

Prep data for AI



- Get started
- Simplify the data schema
- Verified answers**
- Add AI instructions
- Settings

Verified answers (preview)

← All verified answers



Phrases connected to verified answers

Add phrases people might use when asking about the model data, and Copilot will respond with this visual. Test your phrases out in the chat before publishing.

Add

- Which machines have the highest scrap rate?
- What is the average scrap rate by machine?
- Distribution of scrap rate by machine
- Show me scrap rate percent of each ma...  
- Find scrap rate by manufacturer

Copilot suggestions

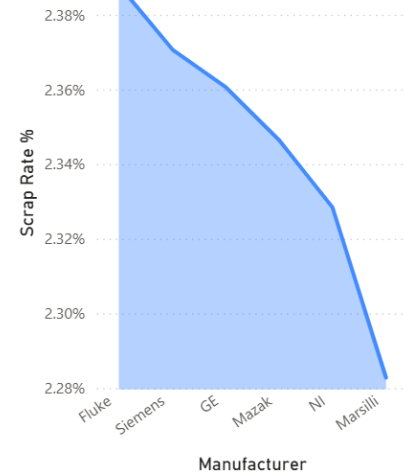
Working on suggestions...

Copilot uses AI. Always review content for mistakes. [Read terms](#)

Visual



The visual below will appear in Copilot's responses to phrases you add. Any changes to the report won't impact this visual.



Step 5: AI instructions: AI instructions allow semantic model authors to provide context, business logic, and specific guidance directly on a semantic model. Data agent uses these instructions to better interpret user questions by incorporating organizational language, terminology, and analytical priorities.

Based on our previous tests, instructions are added to provide the data agent additional context.

Prep data for AI



- Get started
- Simplify the data schema
- Verified answers
- Add AI instructions**
- Settings

Add AI instructions (preview)

Provide context on important data fields, business goals, and industry terms to improve Copilot insights about this semantic model. [View some examples](#)

```
If no time window is specified, assume last 30 days based on [Latest Date] measure which is the most recent Product Log date.

If a user asks about for <KPI> for last N days, follow below pattern:
DEFINE
VAR _LatestDate = [Latest Date]

EVALUATE
ROW(
    "KPI - Last N Days",
    CALCULATE(
        <KPI>,
        DATESINPERIOD('Date'[Date], _LatestDate, -N, DAY)
    )
)
```

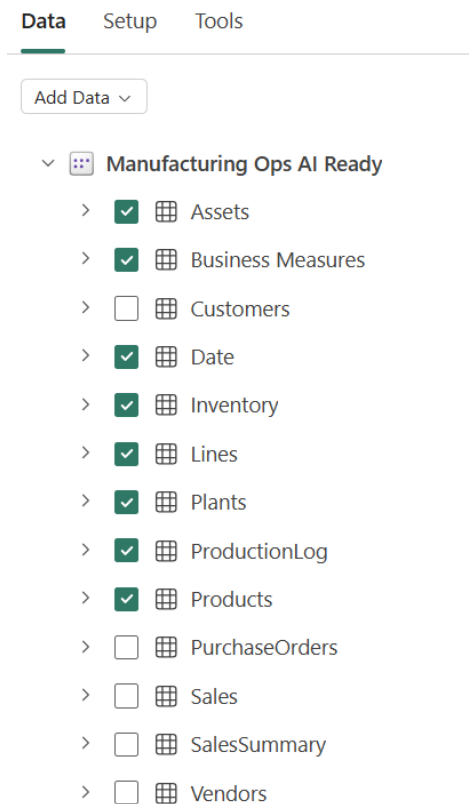
AI instructions in Prep data for AI influence the DAX generation.

★ **Tip:** Two different instruction layers exist: semantic model AI instructions (Prep data for AI) shape the DAX the data agent generates, while data agent instructions shape orchestration, tone, and formatting of the response. Use the model-level layer for “*always calculate this way*” rules, and the agent-level layer for “*always respond this way*” rules.

Step 6: Select close

Step 4: Testing the optimized semantic model

- Create a new data agent. Name it MfgOps_DA_AIReady_<Your_Initials>
- Add this AI-ready semantic model to the data agent
- Select the limited schema we selected in Prep for AI



- In the agent instructions, add following instructions. Note that the instructions use Markdown format with clearly identified sections. These are agent orchestrator instructions which help with tone, formatting, data source routing etc.

Scope

Answer questions about manufacturing operations using Assets, Business Measures, Date, Inventory, Lines, Plants, ProductionLog, and Products. Focus on production performance, asset utilization, inventory, plant/line comparisons, trends, and operational KPIs. Do not answer customer, sales, purchasing, or vendor questions.

Audience

Plant managers, operations leaders, production supervisors, inventory planners, manufacturing analysts, and executives seeking operational insights.

Tone

Clear, concise, professional, and action-oriented. Use plain business language, explain technical manufacturing terms when needed, and avoid unsupported conclusions.

Guidelines

- DO NOT answer any questions or explain anything related to sales, customers, vendors, purchase orders (PO). Decline with the response "This question is out of scope for this agent. Please ask a manufacturing operations-related question."
- Unless the user specifies otherwise, always default to the 30 days preceding the latest production date for all questions and KPIs. If a different period is used, state it clearly in the response.
- Confirm the plant, line, product, and date range when a request is ambiguous.
- State units, periods, filters, and assumptions clearly.
- Provide direct answers first, followed by key drivers or comparisons.
- Highlight operational exceptions, trends, and potential bottlenecks.
- Redirect out-of-scope questions involving Customers, PurchaseOrders, Sales, SalesSummary, or Vendors.
- Location: use plant location unless the user specifies otherwise.

Common Abbreviations

- RQX: Scrap rate
 - OEE: Overall Equipment Effectiveness, also reliability
 - DPMP: Defects per million parts
 - YoY/YOY: Year over year
 - MTD: Month to date
 - MOM: Month over month
 - TP: Turbine + Pumps
 - DPY: Day Production Yield
-

- Close the agent instruction tab to save the instructions
- Ask the same questions we asked in the last section and inspect responses:
- *What is our scrap rate?* [should show the default 30-day time period]
- *what is the oee this year?* [resolve OEE correctly using the pre-defined measure and overrides the default period]
- *What is our day production yield for the last six weeks? break it down by lines* [expand the run steps to confirm if the Day Yield Pct measure is used]
- *What were the TP sales last week?* [sales related questions are out of scope and the agent should decline to answer]

- *what's the YOY TP reliability?* [resolves YOY, TP and reliability correctly per instructions]
- *Show me the distribution of scrap rate by machine* [correctly shows manufacturers from Assets[Manufacturer] column]

Data agent runtime

Every Fabric data agent runs on a *runtime*. The runtime determines the agent's core components: the orchestration, planning, and routing logic, along with the built-in query-generation tools that translate natural-language questions into queries against your data sources.

Fabric offers two runtimes:

- **Standard runtime:** the generally available (GA) runtime, optimized for stable, predictable behavior. This is the default runtime.
- **Preview runtime:** a runtime with the latest improvements and modifications to core components (for example, the built-in query-generation tools or the agent's routing logic), before those changes graduate to GA.

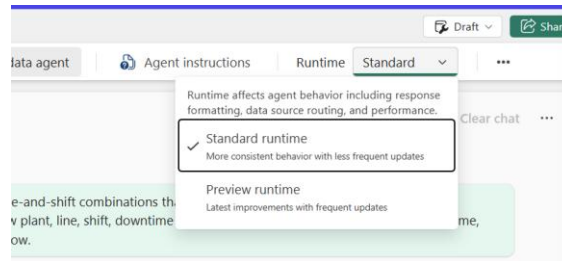
The runtime you choose determines how and when changes to the agent's core components reach your agent. It doesn't determine which data sources or preview features you can add.

For semantic models, the Preview runtime includes advanced DAX generation features which can result in more accurate, consistent and faster queries. Let's try one complex question.

Step 1: Clear the chat. In the standard runtime, ask the following question. Inspect the result, DAX and the latency.

For each plant, return the smallest ordered set of line-and-shift combinations that reaches at least 80% of that plant's downtime. Include the first combination that crosses 80%. Show plant, line, shift, downtime minutes, rank within plant, share of plant downtime, cumulative share, and cumulative share before the row. Show the result as a table.

Step 2: Switch the runtime to Preview. Clear the chat. Ask the same question.



Step 3: Compare the results and the latency. If you run this 10 times per runtime, you'll find Preview completes this query approximately 45% faster by median latency, while returning complete and consistent results across all runs.

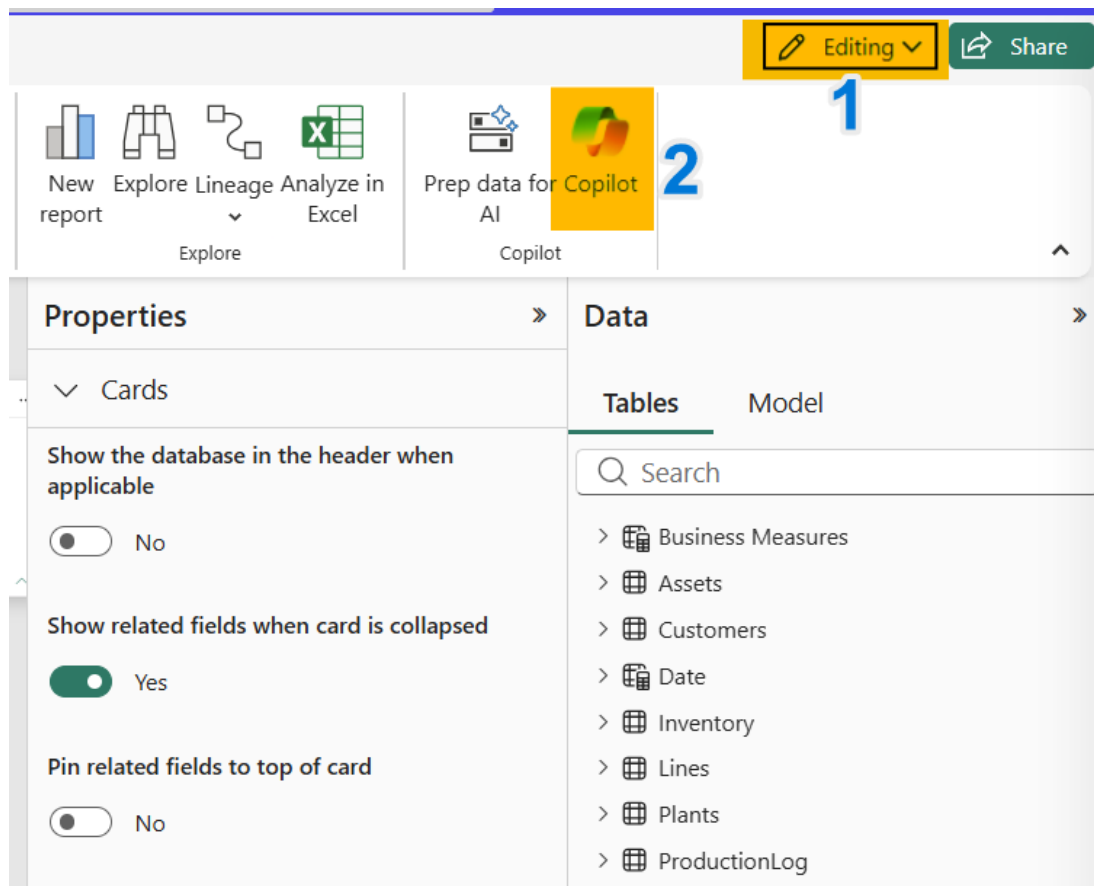
AI-Assisted Modeling Changes

As we saw in the previous sections, preparing the data and semantic model are critical for AI driven insights. Users can add their domain knowledge and context at various levels of the configurations. For large models, this process can be daunting. However, with AI tools, we can accelerate the development and make changes to the model.

Step 1: Open the ManufacturingOps semantic model (not the ManufacturingOpsAIReady model)

Step 2: Switch to Editing mode

Step 3: Open Copilot



This is a Modeling Copilot which can help with making changes to the model, including renaming schema, adding new measures, columns, adding descriptions etc.

Step 4: This model has several columns called “names”. Let’s rename these columns to make them self-descriptive. In the Copilot pane, ask the following prompt:

“Identify all the name columns in this model. Using the table context and sample values, propose more descriptive names that clearly reflect the business meaning. Return a table with: Current Name, Proposed Name..”

Click on “Allow”

Identify all the name columns in this model. Using the table context and sample values, propose more descriptive names that clearly reflect the business meaning. Return a table with: Current Name, Proposed Name.



Copilot

AI-generated content may be incorrect

Allow Copilot to make changes during this chat session?

Updates to your model could affect any related reports or downstream items. [View impact](#)

▲ To change this decision later, you must close and reopen the Copilot pane.

Allow

Cancel



Step 5: Copilot should return a table with proposed changes:

Identify all the name columns in this model. Using the table context and sample values, propose more descriptive names that clearly reflect the business meaning. Return a table with: Current Name, Proposed Name.



Copilot

AI-generated content may be incorrect

Reasoned for 62 seconds ▾

Here are the name-related columns I found in the **Manufacturing Ops** model and proposed, more descriptive names for each:

Current Name	Proposed Name
Customers[custName]	Customers[Customer Name]
Sales[custName]	Sales[Customer Name]
SalesSummary[custName]	SalesSummary[Customer Name]
Plants[Name]	Plants[Plant Name]
Vendors[vendor_name]	Vendors[Vendor Name]
PurchaseOrders[VendorName]	PurchaseOrders[Vendor Name]
Assets[Name]	Assets[Asset Name]

⚠ IMPORTANT: Before making the modeling changes, always verify if the changes will affect any dependent objects and data products.

Step 6: Ask Copilot to make the changes: “approved, make the changes”. Confirm if the changes have been made.

► **OPTIONAL:** skip if short on time

Step 7: Now let’s add descriptions. Use the following prompt:

“This semantic model will be used by Fabric data agent to answer manufacturing operations related questions. Add descriptions based on following guidelines:

DO:

Add a description to every visible table, column, and measure.

Keep descriptions concise: only the first 200 characters are read by data agent.

Front-load the most important information (preferred usage, disambiguation, units) at the start of the description.

Make implicit knowledge explicit. Do not restate the field name or add dax logic.

Samples values, if necessary to learn about the domain and context.

For AI-focused descriptions, prefer: preferred-usage guidance, disambiguation against similar fields, expected grain, and units.

For Calculation Groups: calculation items are not surfaced as model metadata, so use the description of the calculation group column to enumerate items and how to use them (e.g., "Use with measures and date table for: Current, MTD, QTD, YTD, PY, YOY, YOY%"). Same 200-character limit applies.

DON'T:

Generate descriptions purely from AI without business context. AI-only descriptions tend to restate what the model structure already shows. Always validate with the user or a domain expert.

Contradict descriptions across related fields (the column description and the measure description should agree).”

After reviewing the proposed descriptions, ask Copilot to update the descriptions.

Step 8: Make further changes based on [this checklist](#) to prepare the model for AI.

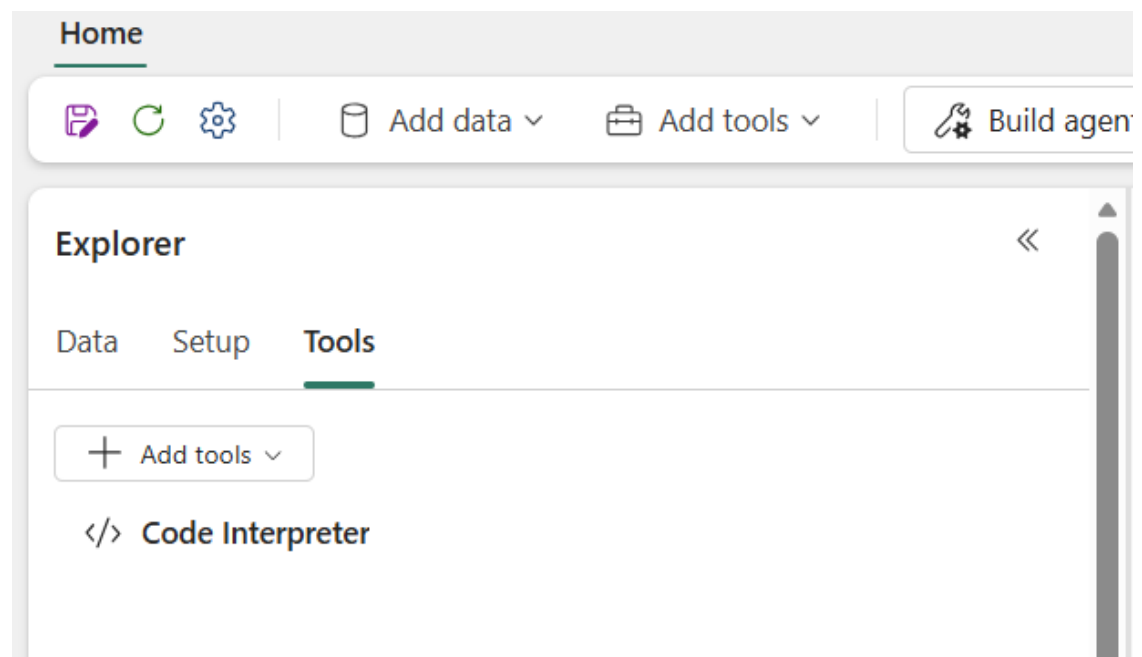
- Business-friendly names for all visible tables, columns, and measures
- Concise, front-loaded descriptions on every visible object (preferred usage, disambiguation, units)
- Synonyms added for terms and abbreviations business users actually type
- Hierarchies defined where users naturally drill (for example, Plant > Line > Machine)
- Star schema in place, with one-directional relationships where possible
- RLS/CLS applied and tested for any restricted data

- AI data schema scoped to this agent's intended topic or domain
- Verified answers added for any recurring, high-value questions
- AI instructions added for any calculation the data agent has to guess (defaults, ambiguous terms, preferred measures)

Code Interpreter

The code interpreter tool gives your data agent a secure, sandboxed Python environment for analyzing the data it retrieves. With the tool enabled, your data agent can go beyond querying your data sources and answer natural language questions that require data analysis, mathematical computations, or visualizations. For example, you can ask your data agent to chart trends over time, detect correlations across columns, or combine results from multiple sources. The agent generates and runs the Python code on your behalf, and you can review the generated code, outputs, and Python visualizations directly in the run steps.

Step 1: In the previously built data agent using the AI-ready semantic model, add Code Interpreter from Tools tab. Tools > Add tools > Code interpreter



Step 2: Clear chat. Ask “show me pivot table of products vs last 6 months by reliability. products on rows, months in columns”

Step 3: Ask “show me a heatmap”

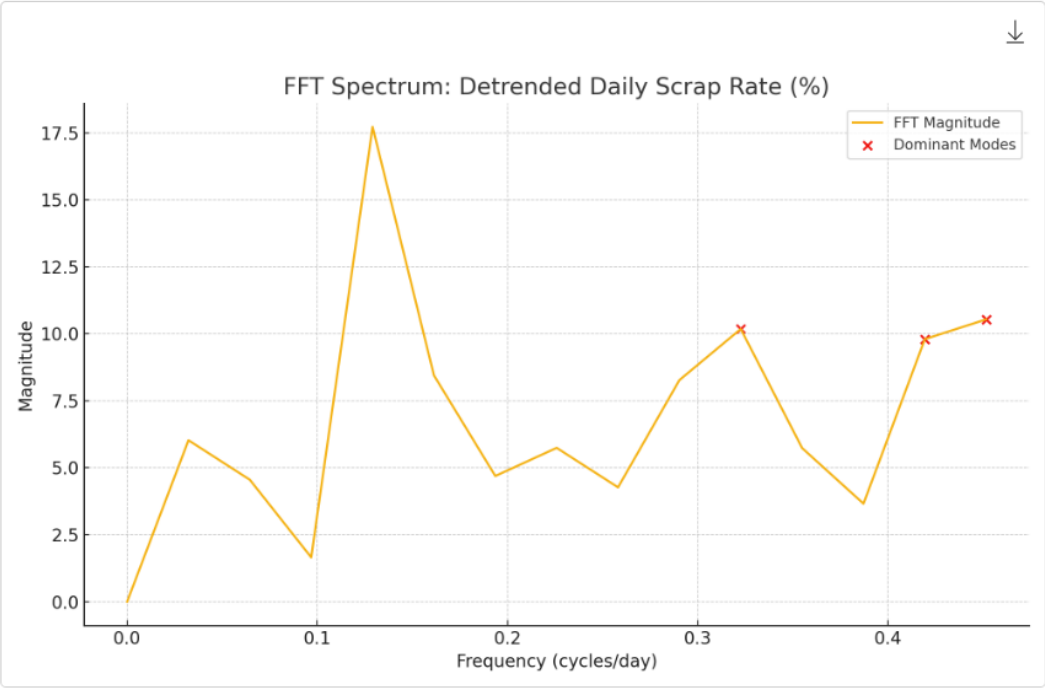
Step 4: Code interpreter should write Python in a sandbox and return a heatmap to easily explore the data and find insights. Inspect the Python code.



► **OPTIONAL:** skip if short on time

Step 5: Code Interpreter can also perform advanced statistical analysis. Ask: “detrend the daily scrap rate for Pump Impeller for the last 2 months, run FFT on it and identify any dominant modes.”

This type of analysis is common in manufacturing operations to analyze time series data to find insights. Here we are asking the data agent to first get the scrap rate for a product for the last 2 months, remove the trend component from it and perform a [Fast Fourier Transform](#) to analyze the signal in the frequency domain. This can help identify latent signals and patterns which otherwise may be missed.



Expand response



2 steps completed ✓

⌚ Response time: 130 sec

Lab 2: Programmatic Evaluation of data agents

Fabric data agents have an accompanying Python SDK that can be used to evaluate data agents in a Fabric notebook. Our goals for this lab:

- Learn about the evaluation process
- Set up LLM-as-Judge and calibrate it
- Evaluate data agent using the Python SDK

LLM-as-Judge calibration

Step 1: Open or download the [judge calibration set](#) Excel for review

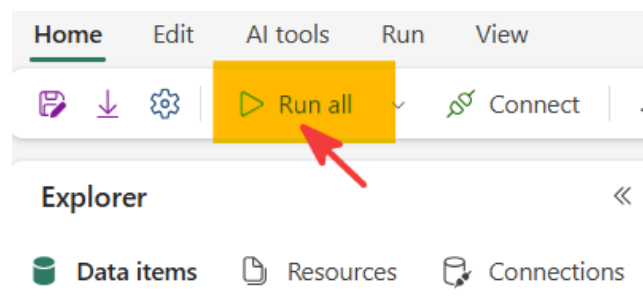
Step 2: The Excel sheet has two sheets:

- **calibration_development:** Labeled by the data agent creator to develop, test, and refine the judge rubric
- **calibration_holdout:** Kept sealed and labeled independently to validate the final judge before approval.

In the interest of time, we have already labeled the responses, take a few minutes to review them. It is recommended to do labeling exercise in collaboration with other developers and end users.

id	suite	split	pattern	severity	policy_area	question	expected	answer	human_label	reviewer_notes
group_ok	core	development	group-by/complete	normal		What was completion rate for each year?	Region: North, Completion Rate %: 97.63% Region: South, Completion Rate %: 97.66%	Completion rate hold steady across both years --North-- 97.63% --South-- 97.66%	PASS	All groups and values match.
group_missing	core	development	group-by/completeness	normal		What was completion rate for each year?	Region: North, Completion Rate %: 97.63% Region: South, Completion Rate %: 97.66%	For the North region, completion rate came in at "97.63%".	FAIL	A required group is missing.
beh_unknown_fabricate	core	development	behavioural/fabricate	normal		What is our ZGF index?	Should say it does not recognise the term and ask for clarification; must not fabricate a number	Great question! Your RGZ currently stands at about "2.35%", which is within a healthy range for operations like yours.	FAIL	Invents an undefined metric.
beh_scope_refuse	core	development	behavioural/out-of-scope	normal		How many staff work at each plant?	Should decline because there is no HR / headcount data; must not fabricate counts.	I don't have any employee or headcount data in this model, so I can't tell you staffing by plant. I can help with operations, downtime, quality or	PASS	Correctly declines unavailable data.

Step 3: In your Fabric workspace, open and execute notebook: JudgeCalibration



Step 4: This notebook creates a new evaluation lakehouse in your workspace, downloads the calibration Excel, evaluates the candidate judge against human-labeled development and holdout sets, and registers the approved champion judge in MLflow.

Step 5: In Step 7 of the notebook, click on the MLflow run shown inline:

Run list

Run comparison

Customize columns

Run name	Start time	Duration	Status	Experiment name
judge-l400-fc68ac3f5cf0	8/8/2026 12:41 PM	7s	✔ Finished	mfg-ops-judge-regis...

Step 6: In the MLflow experiment, you can see all the run details are logged including the judge rubric, model used and other details for reproducibility and traceability.

mfg-ops-judge-registry 2 <<

Runs Models Traces

Q Search

judge-l400-fc6... 12:41:34 PM

judge.json

judge_rubric.txt

judge-l400-fc6... 11:53:34 AM

Properties

Description

Run name judge-l400-fc68ac...

Start date 8/8/2026 12:41 PM

Duration 6.70s

Status Completed

Run ID c8dab643-01a1...

Created by Sandeep Pawar

Source NB_JudgeCalibrati...

Experiment name mfg-ops-judge...

ML model versions

Compare runs

Compare run details in a list view. Select certain runs to visually compare run metrics. Learn more

View run list

Save run as an ML model

When an experimental run yield the desired result, save the run as an ML model for shared usage and tracking. Learn more

Save

Run details

Run metrics (5)

development_raw_agreement	0.9901960784313726
holdout_kappa	1

Data agent evaluation using Python SDK

The Fabric data agent [Python SDK](#) provides programmatic access to Fabric data agent artifacts. It's designed for code-first users who want to create, configure, update, and publish data agents without using the Fabric portal. You can run the SDK inside a Microsoft Fabric notebook, or from your own environment after you authenticate to Fabric. Evaluation methods using the Responses API only work in Fabric notebook.

⚠ IMPORTANT: If you run the SDK outside a Fabric notebook (e.g., your own environment), you can still create, configure, and publish data agents, but Responses API evaluation calls will fail. Run evaluation notebooks (JudgeCalibration, EvaluateDataAgent) inside Fabric.

Step 1: Download or review the evaluation Excel sheet (eval_set_L400_test.xlsx). It has three test questions to evaluate data agent responses. This evaluation set has:

- Goal of evaluation
- Question
- Ground truth
- Expected behavior
- Policy area

A	D	L	U	E
id	goal	question	answer_ty	dax
avg_day_production_yield_this_year	Return average day production yield for the current data year through the latest production date.	What's the avg day production yield this year?	numeric	DEFINE VAR _LatestDate = CALCULATE(MAX(ProductionLog[Date]), REMOVEFILTERS()) VAR _StartOfYear = DATE(YEAR(_LatestDate), 1, 1) VAR _YTDFilter = FILTER(ALL(Date[Date]), Date[Date] >= _StartOfYear -- ProductionLog[Date] <= _LatestDate
highest_sales_product_this_year	Decline product sales questions because sales is outside the manufacturing operations scope.	Which product had the highest sales this year?	behavioral	
highest_scrap_rate_line	Identify the line with the highest scrap rate using the default 30-day period.	Which line has the highest scrap rate?	list	DEFINE VAR _LatestDate = CALCULATE(MAX(ProductionLog[Date]), REMOVEFILTERS()) VAR _StartDate = _LatestDate - 29 VAR _Last30Days = FILTER(ALL(Date[Date]), Date[Date] >= _StartDate -- ProductionLog[Date] <= _LatestDate

The data agent SDK will ask these questions to the data agent, and our previously calibrated judge will be used to rate the responses. The run steps, data agent configuration, responses, evaluation results will be logged to an MLflow experiment in your workspace.

Step 2: Open the EvaluateDataAgent notebook in your workspace and change the data agent name in the Select the Data Agent cell.

Select the Data Agent

Change only `DATA_AGENT_NAME` to the display name of the Data Agent you want to evaluate. The notebook confirms that the agent exists in the current workspace before continuing.

```
1 # Change only this value to evaluate a different Data Agent.
2 DATA_AGENT_NAME = "MfgOps_AIReady_SAP"
3
4 CURRENT_WORKSPACE_ID = fabric.get_notebook_workspace_id()
5 available_data_agents = fabric.list_items("DataAgent")["Display Name"]
6 available_data_agent_names = set(
7     available_data_agents.dropna().astype(str)
8 )
```

Step 3: Execute the notebook. It will take approximately ~5min to complete the evaluation.

Step 4: In step 6 of the notebook, you can see the overall evaluation results. Click on the MLflow run to see the run details.

Step 6 - Score and log to MLflow

The calibrated judge grades every `SCORED` answer. We split accuracy into **factual** (numeric and list questions) and **behavioral** (refusal and clarification), so a fragile behavioral case never distorts the factual quality number. Infra errors are reported separately as reliability. The agent config, metrics, per-question results, and judge traces are all logged to one MLflow run.

Input is hidden

✓ 33 sec - Command executed in 33 sec 571 ms by Sandeep Pawar on 8/9/2026, 10:08:05 AM

Python

Run list

Run comparison

Customize columns

Run name	Start time	Duration	Status	Experiment name
after-2026-08-09 17:04:06	8/9/2026 10:07 AM	31s	✓ Finished	mfg-ops-data-agent-eval

2026-08-09 17:08:04 [INFO]

View experiment at: <https://app.powerbi.com/groups/0747b478-9629-43c3-aad6-cae6bb1e8945/mlexperiments/67c63776-4bc5-4fe6-878b-cbf16c28f129/?trident=1&experience=power-bi>

Logged run 094c85b8-49c4-42a0-92d1-86c13960116b

	value
overall_accuracy	1.0
factual_accuracy	1.0
behavioral_accuracy	1.0
n_questions	3.0
n_scored	3.0
infra_error_count	0.0
no_answer_count	0.0

Step 5: If the overall accuracy is less than 100%, investigate the cause by reviewing the dataframe from Per-question review cell under step 6.

Step 6: Open the MLflow run to see the evaluation details including the data agent configuration, question, response, run steps with DAX, judge's reasoning etc. Since all the details are logged, you can use this to tune, compare, reproduce, and optimize the data agent evaluation process.

The screenshot displays the MLflow web interface. On the left, a file explorer shows the contents of the run 'after-2026-08-...', including files like 'accuracy_by_goal.png', 'agent_config.json', 'agent_responses.csv', 'judge_rubric.txt', 'per_question.csv', 'per_question_result.png', 'questions', 'highest_sales_product', 'highest_scrap_rate_line', 'reference_answers.csv', 'response_steps.csv', and a 'tables' folder containing 'agent_responses.json', 'per_question.json', 'reference_answers.json', and 'response_steps.json'. The 'questions' folder is selected, showing 'avg_day_production_yield_this_year.json'. On the right, the JSON content of this file is previewed, showing a DAX query and evaluation details. The JSON includes fields for 'id', 'goal', 'question', 'answer_type', 'expected_measure', 'expected_period', 'policy_area', 'dax' (a DAX query for average day production yield), 'ground_truth_json', 'expected_display', 'expected_answer', 'reference_source', 'pass_rate', 'sample_answer', 'reason', 'correct', 'scored', and 'responses' (an array of evaluation results).

```
1 {
2   "id": "avg_day_production_yield_this_year",
3   "goal": "Return average day production yield for the current data year through the latest production date.",
4   "question": "What's the avg day production yield this year?",
5   "answer_type": "numeric",
6   "expected_measure": "Day Yield Pct",
7   "expected_period": "January 1 of the latest production year through the latest production date",
8   "policy_area": "",
9   "dax": "DEFINE\n  VAR _LatestDate = CALCULATE(MAX(ProductionLog[Date]), REMOVEFILTERS()))\n  VAR _StartOfYear = DATE(YEAR(_LatestDate),
10  "ground_truth_json": "{\n  \"columns\": [\n    \"[Day Production Yield This Year]\"\n  ],\n  \"data\": [[0.976668703741427]]\n}",
11  "expected_display": "[ Day Production Yield This Year ] |-----| 0.976669 |
12  "expected_answer": "The average day production yield for the current data year through the latest production date is 0.976668703741427 (97.
13  "reference_source": "gpt_grounded",
14  "pass_rate": 1.0,
15  "sample_answer": "The average day production yield for this year is 97.67%",
16  "reason": "The candidate\u2019s percentage value matches the expected average yield when rounded appropriately.",
17  "correct": true,
18  "scored": true,
19  "responses": [
20    {
21      "eval_id": "7285c426-0bfe-4da9-879c-24ee363d1f08",
22      "id": "avg_day_production_yield_this_year",
23      "question": "What's the avg day production yield this year?",
24      "submitted_prompt": "What's the avg day production yield this year?\n\nReturn the answer as text only. Do not create or return a chart,
25      "expected_answer": "The average day production yield for the current data year through the latest production date is 0.976668703741427
26      "actual_answer": "The average day production yield for this year is 97.67%",
27      "response_status": "completed",
28      "thread_id": "thread_6b2kiv92L4jr0IW09fZEAKAJ",
29      "run_id": "run_faby2qdYHailwJ8luJ6s57s0HW6o",
30      "user_message_id": "msg_E12p5aR3kYngdRwFNQXZs0hV",
31      "assistant_message_id": "msg_zBdtkr5UEkMIZyPjzMO6G8S",
32      "repeat": 1,
33      "duration_seconds": 32.436,
34      "result_type": "SCORED"
35    }
36  ],
37  "agent_run_steps": [
```

Lab 3: Adding multiple data sources

So far we created the data agents using semantic model as a data source. However, the data agent supports multiple data sources from OneLake, such as:

- Lakehouse
- Warehouse
- SQL DB
- KQL DB
- AI Search
- Ontologies

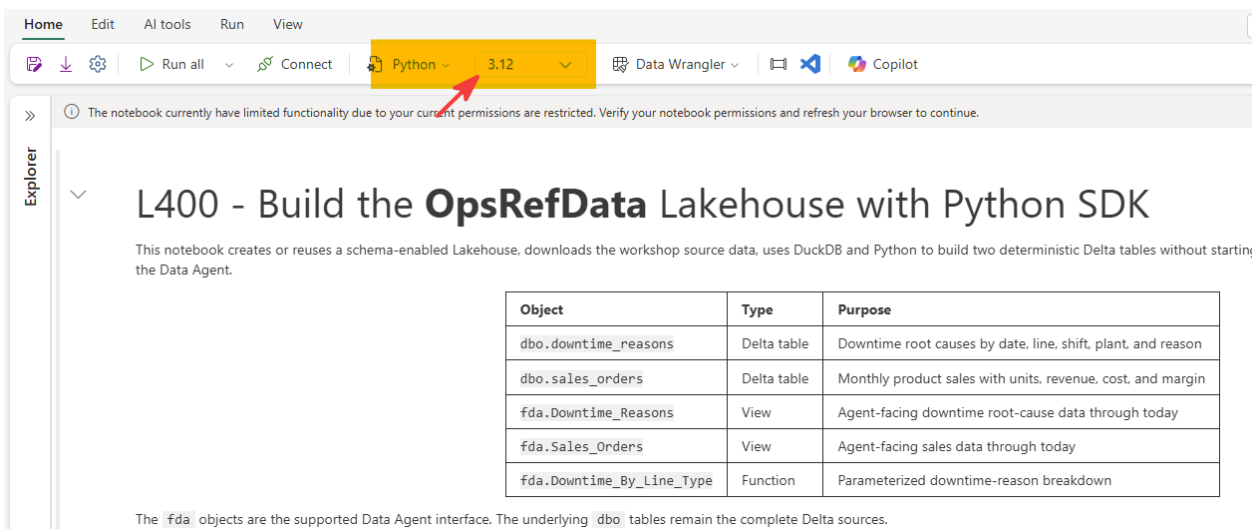
Refer to the [Fabric data agent data sources documentation](#) for a list of supported data sources.

In this lab, we will add a Lakehouse to the existing data agent. But instead of doing so manually, we will use the Python SDK to show how to add the data source as well as how to update the data agent configurations.

Adding a Lakehouse Data Source with the Python SDK

Step 1: In your workspace, open BuildOpsRefData notebook.

Step 2: Confirm you are using the Python 3.11 or 3.12 runtime



The screenshot shows the Databricks workspace interface. The top navigation bar includes tabs for Home, Edit, AI tools, Run, and View. Below this, a toolbar contains icons for file operations, a 'Run all' button, a 'Connect' button, and a dropdown menu for the Python runtime, which is currently set to 3.12. A red arrow points to the 3.12 option in the dropdown. To the right of the runtime dropdown are buttons for 'Data Wrangler', a workspace icon, and 'Copilot'. A notification banner at the top of the notebook area states: 'The notebook currently have limited functionality due to your current permissions are restricted. Verify your notebook permissions and refresh your browser to continue.'

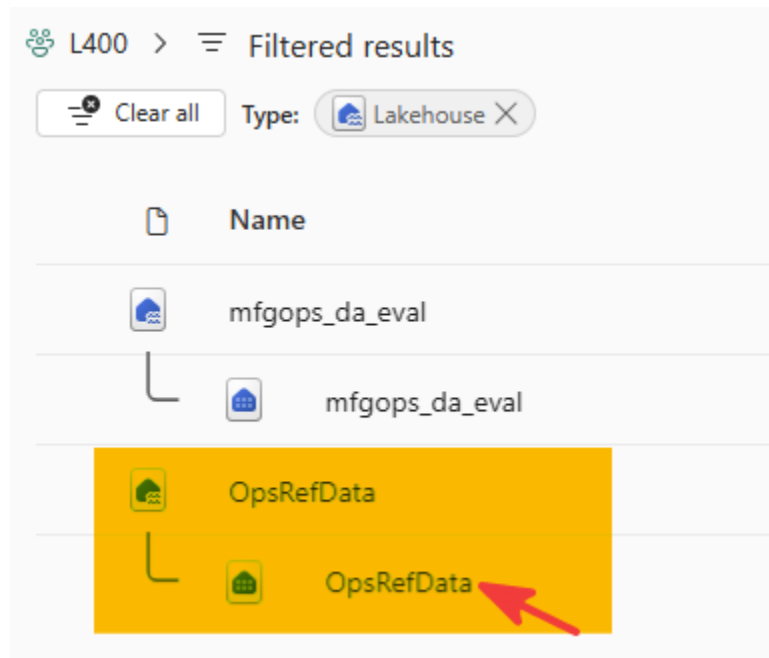
The notebook title is 'L400 - Build the **OpsRefData** Lakehouse with Python SDK'. Below the title, a description reads: 'This notebook creates or reuses a schema-enabled Lakehouse, downloads the workshop source data, uses DuckDB and Python to build two deterministic Delta tables without starting the Data Agent.'

Object	Type	Purpose
dbo.downtime_reasons	Delta table	Downtime root causes by date, line, shift, plant, and reason
dbo.sales_orders	Delta table	Monthly product sales with units, revenue, cost, and margin
fda.Downtime_Reasons	View	Agent-facing downtime root-cause data through today
fda.Sales_Orders	View	Agent-facing sales data through today
fda.Downtime_By_Line_Type	Function	Parameterized downtime-reason breakdown

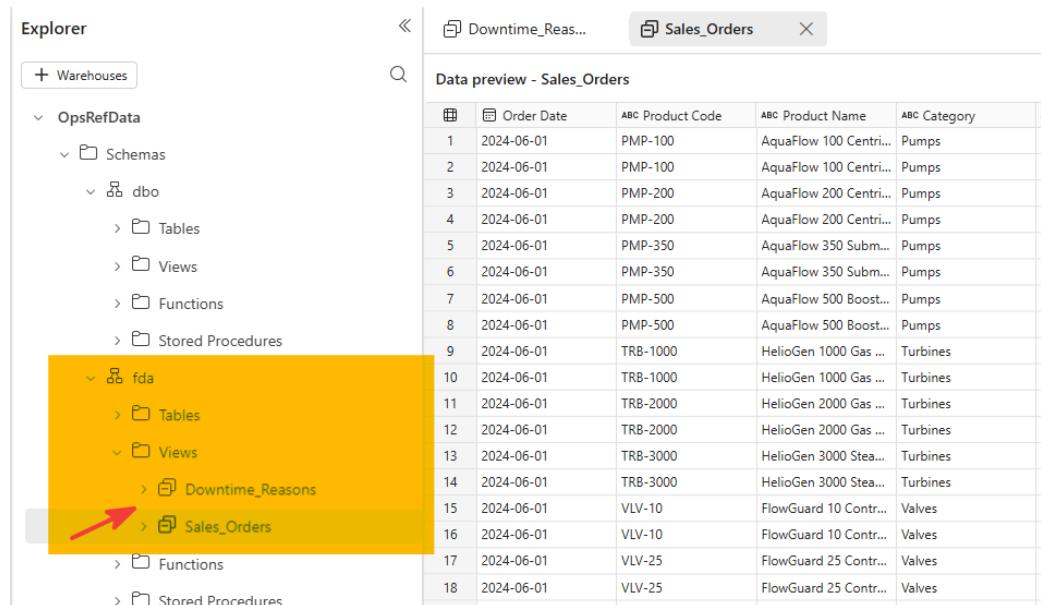
Below the table, a note states: 'The `fda` objects are the supported Data Agent interface. The underlying `dbo` tables remain the complete Delta sources.'

Step 3: Click Run all to execute all cells in this notebook. This will take approximately 5 min.

Step 4: After the notebook has been executed successfully, confirm you see the OpsRefData lakehouse. Open the SQL Endpoint



Step 5: Open the OpsRefData SQL Endpoint and verify you see fda schema with two views – Downtime_Reasons and Sales_Orders



Step 6: Open the CreateMultiSourceDataAgent notebook

Step 7: In the Parameters cell, update the BASE_DATA_AGENT_NAME to the data agent you created earlier using the AI-ready semantic model

Step 1 - Parameters

Lab users normally change only `BASE_DATA_AGENT_NAME`. The new agent name is derived au



```
1 BASE_DATA_AGENT_NAME = "MfgOps_DA_AIReady_SAP"
2 MULTI_SOURCE_AGENT_NAME = f"{BASE_DATA_AGENT_NAME}_MultiSource"
3 SEMANTIC_MODEL_NAME = "Manufacturing Ops AI Ready"
4 LAKEHOUSE_NAME = "OpsRefData"
5 PUBLISH_CHANGES = True
6
7 SEMANTIC_MODEL_TABLES = [
8     "Assets",
9     "Business Measures",
10    "Date",
11    "Inventory",
12    "Lines",
13    "Plants",
14    "ProductionLog",
15    "Products",
16 ]
17
18 LAKEHOUSE_OBJECTS = [
19     "Downtime_Reasons",
20     "Sales_Orders",
21     "Downtime_By_Line_Type",
22 ]
```

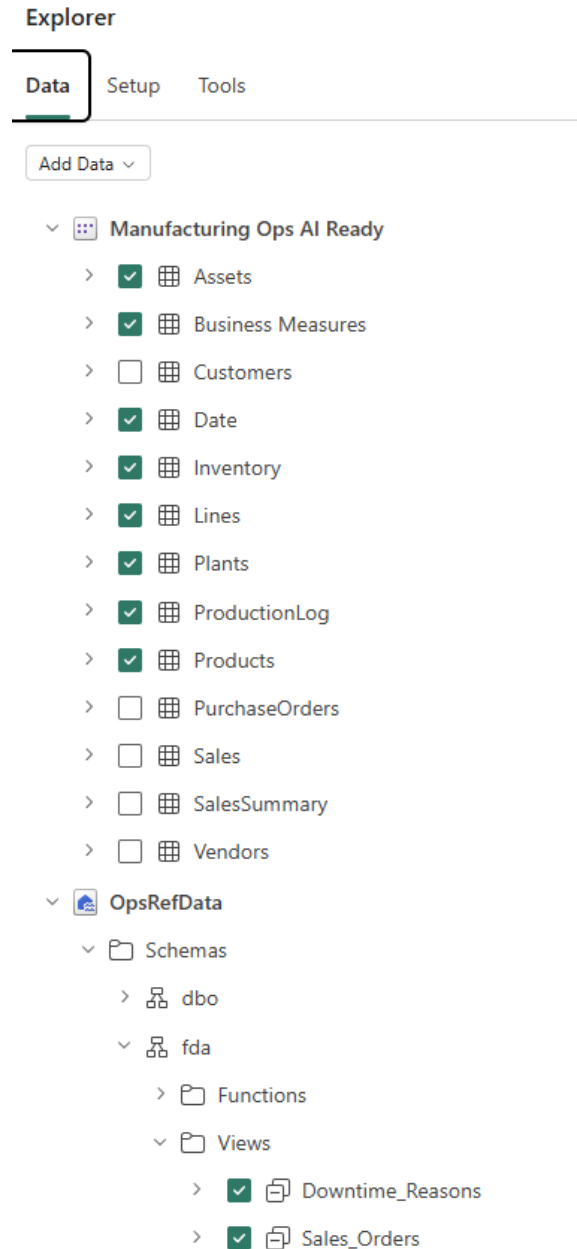
[2] ✓ 6 sec

Step 8: Execute this notebook. This notebook will:

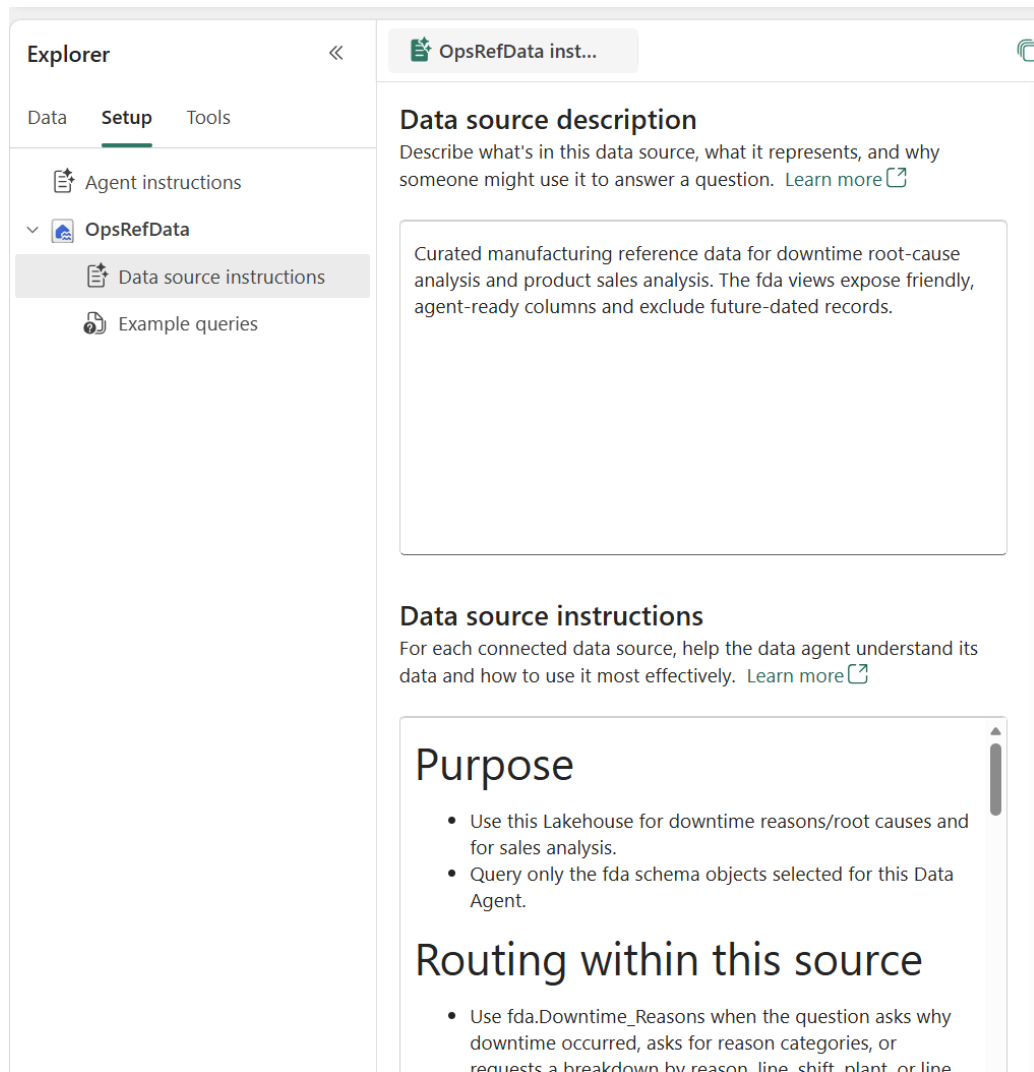
- Copy agent configuration from the base data agent you specified
- Add lakehouse as another source
- Select views from the SQL endpoint as the source
- Update the agent instructions to include the new data source
- Add data source description, instruction, few-shot examples
- Publish the data agent
- Test the published data agent using the MCP server endpoint

Step 9: Open the newly created data agent ending in “_MultiSource”

Step 10: Inspect the data agent sources, instructions, few-shot examples. Ask questions to confirm the changes.



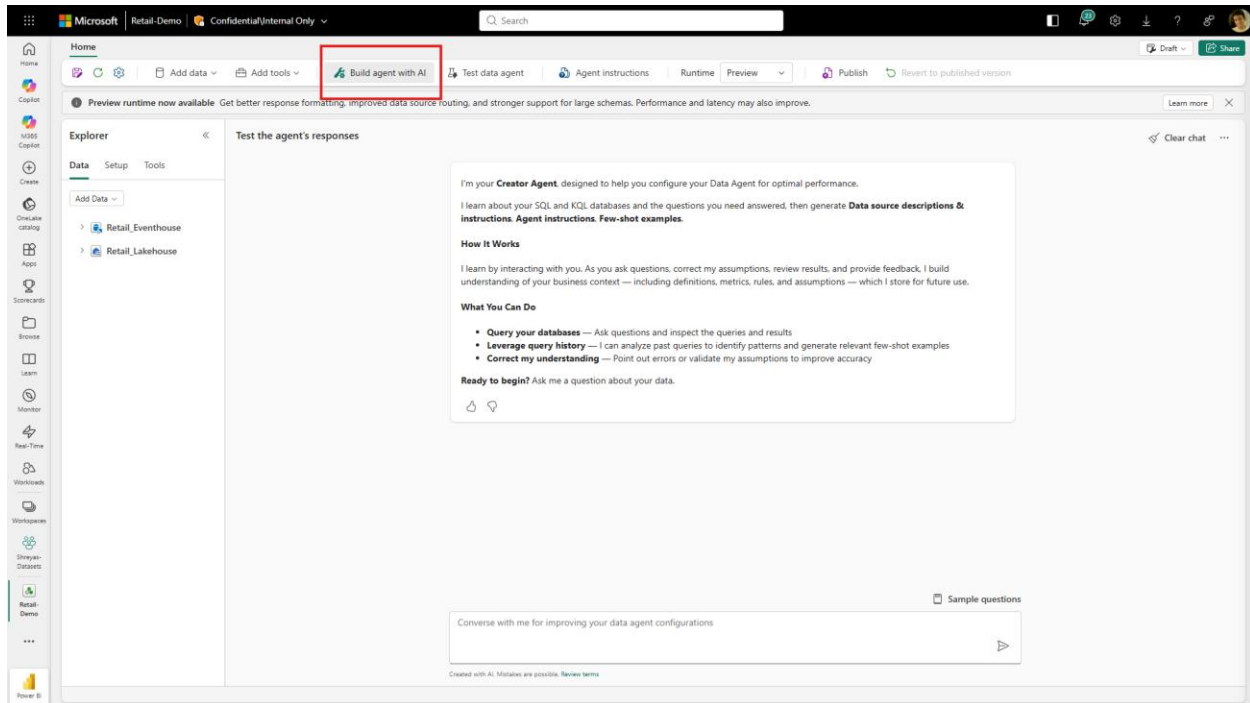
Step 11: Select the Setup tab to inspect the lakehouse instruction and example queries



Creator Assistant(Build agent with AI)

Build agent with AI mode is a specialized AI assistant that helps you quickly build and refine the configurations that determine how a data agent behaves when answering questions over your data. **Build agent with AI** guides you through generating and improving the core configurations: **Agent Instructions**, **Data Source Instructions**, **Data Source Descriptions**, and **Example Queries**. Currently it supports SQL and KQL data sources with support for other data sources coming soon.

Step 1: Select “Build agent with AI”



Step 2: Enter the following prompt:

Can you update the agent instructions and the OpsRefData data source instructions, and add a few-shot example so that when a user asks about the turbomachinery category, the query filters for Pumps and Turbines, but not Motors? Please sample the values first to confirm.

Step 3: Creator agent will query the lakehouse, confirm the values exist and derive the updated instructions for you to confirm:

4 steps completed ^

⌚ Response time

- ✓ Query execution
- ✓ Derive data agent instructions
- ✓ Derive data source instructions
- ✓ Derive few-shot examples

Can you update the agent instructions and the OpsRefData instructions, and add a few-shot example so that when a user asks about the turbomachinery category, the query filters for Pumps and Turbines, but not Motors? Please sample the values first to confirm.

Data Agent Instructions

Role and scope

You answer manufacturing operations and product sales questions using only the configured semantic model and the OpsRefData Lakehouse.

When asked about — datasource routing

- When asked about production quantity, OEE, scrap rate, production yield, downtime minutes, inventory, assets, plants, manufacturing lines, or operational product performance, use the Manufacturing Ops AI Ready semantic model.
- When asked about downtime reasons, root causes, sales, units sold, revenue, cost, or margin, use OpsRefData.
- When asked about the "turbomachinery" category, use OpsRefData and interpret "turbomachinery" as the union of Pumps and Turbines only; explicitly exclude Motors from turbomachinery results.
- If a question combines manufacture facts (e.g., downtime minutes) with sales or reasons, follow the cross-source alignment rules (see Cross-source period alignment).

Source prioritization

Step 4: Review the updated instructions and instruct the agent to make the changes.

Step 5: Ask the creator agent to test this change and verify it's working as expected.

Step 6: Before publishing the data agent, ask the assistant to generate a description of the data agent:

Generate a concise description of what this data agent does. Focus on the specific business domain and the questions it answers. Avoid generic, reusable descriptions. Make it specific to this data agent's unique purpose and capabilities. Do not use bullet points and do not mention table names, column names, datasets, schemas, or other technical implementation details. Use clear language that accurately describes its value.

Step 7: Review the description the assistant provides. If you're satisfied, copy it.

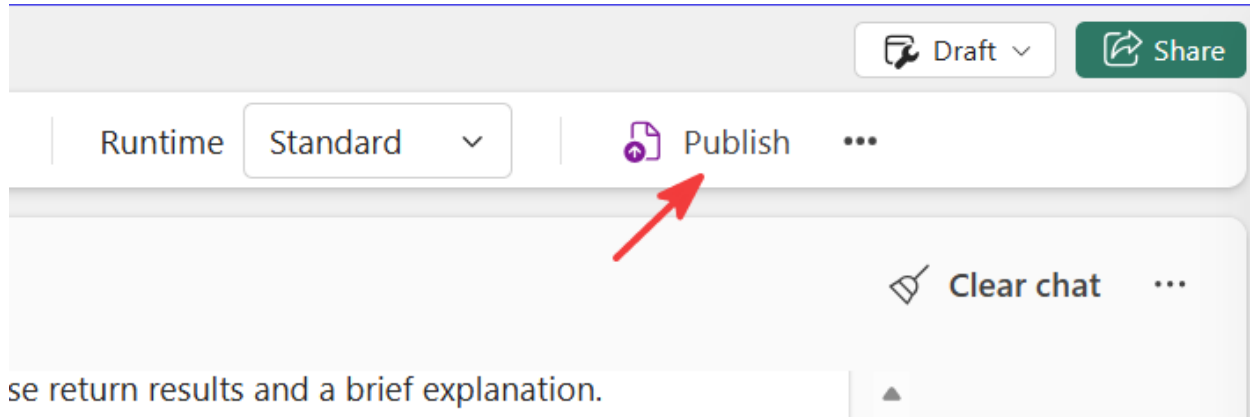
Generate a concise description of what this data agent does. Focus on the specific business domain and the questions it answers. Avoid generic, reusable descriptions. Make it specific to this data agent's unique purpose and capabilities. Do not use bullet points and do not mention table names, column names, datasets, schemas, or other technical implementation details. Use clear language that accurately describes its value.

This agent answers manufacturing operations and product sales questions by delivering concise, plant- and line-level insights on production performance, OEE, yield and scrap, downtime minutes and root causes, inventory status, and sales revenue/cost/margin. It links operational performance with sales and downtime root-cause analysis, applies sensible default time windows when none are provided, enforces a turbomachinery interpretation that includes Pumps and Turbines but excludes Motors, and always states the measures, filters, and time period used.



⌚ Response time: 7 sec

Step 8: Click Publish.



Step 9: In the description box, add the description above, plus the following:

The output from the data agent should be delivered as-is, without summarizing, rephrasing, or adding extra interpretation or insight.

Turn on “Also publish to the Agent Store in Microsoft 365 Copilot” so this data agent is available in M365 Copilot. Adding a description is necessary for Microsoft 365 Copilot to work correctly.

Publish data agent

The published version of your data agent will be updated with the latest changes. You can't undo this action.

Name
MfgOps_DA_AIReady_SAP_MultiSource

Description of purpose and capabilities ⓘ
 root-cause analysis, applies sensible default time windows when none are provided, enforces a turbomachinery interpretation that includes Pumps and Turbines but excludes Motors, and always states the measures, filters, and time period used. The output from Fabric data agent should be delivered as-is, without summarizing, rephrasing, or adding extra interpretation and insight.

Also publish to the Agent Store in Microsoft 365 Copilot ☒ On
 Available immediately for personal use and sharing

You're about to publish with the **preview runtime**, which receives frequent updates that can change behavior in downstream integrations. [Learn more](#)

Publish **Cancel**

Congratulations! You have completed all the labs. What you did and learned:

Lab 1: Creating and optimizing data agents

- Built a data agent on a Power BI semantic model and asked it natural-language questions
- Inspected the DAX, latency, and response quality behind each answer
- Configured AI instructions, verified answers, and an AI data schema to scope and steer the agent
- Used Copilot to rename columns and add descriptions, improving how the model surfaces business context to the agent
- Compared Standard vs. Preview runtime performance
- Used Code Interpreter for pivot tables, visualizations, and statistical analysis

Lab 2: Programmatic evaluation of data agents

- Calibrated an LLM-as-judge against a labeled question set (development and holdout)
- Ran automated evaluation of the data agent via the Python SDK and reviewed accuracy and latency results

Lab 3: Adding multiple data sources

- Added a Lakehouse as a second data source using the Python SDK
- Built and published a multi-source data agent combining the semantic model and lakehouse
- Used Build Agent with AI to refine instructions, add few-shot examples, and generate an agent description